DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING • PES UNIVERSITY, BENGALURU

UE24CS395: Database Management Systems Mini Project • Academic Year 2025–2026

Technology Justification & Architecture Synthesis

Evaluation Matrix Row 5: Technology Justification • Weightage: 1 Mark

Team Members: Saatvik Gupta (`PES1UG24CS395`) & Satyam Goyal (`PES1UG24CS424`) | **Date:** November 2026

1. Problem Statement & Architecture Rationale

Modern criminal intelligence platforms handle two distinct categories of data: (1) highly structured, legally accountable operational data (such as chain-of-custody transfer records, case assignments, warrants, and suspect associations), and (2) polymorphic, semi-structured forensic data (including witness transcripts, CCTV telemetry, phone extraction dumps, and financial transaction streams).

Attempting to store all data within a single database paradigm leads to clear architectural compromises. A relational-only approach produces sparse tables with excessive null fields, schema migration friction, and difficulty storing nested telemetry. Conversely, a document-only approach lacks native ACID transactional guarantees, foreign key cascade enforcement, and kernel-level role-based privileges required for evidentiary integrity in court proceedings. We resolved this challenge by implementing a **polyglot persistence architecture**:

Tier / Technology	Core Responsibilities	Primary Justification & Selected Trade-Offs
Relational Tier PostgreSQL 16 *(SQLite Fallback)*	22 normalized tables: officers, cases, people, evidence, chain-of-custody logs, M:N association bridges.	**ACID Transactions & Foreign Key Enforcement:** Guarantees non-repudiation and prevents orphaned exhibits. Stored procedures and triggers automate audit logging. We include an automated SQLite fallback for offline viva evaluation.
Document & Vector Tier MongoDB 7.0	Polymorphic `investigation_documents`, CCTV logs, forensic phone dumps, 64-dim semantic embeddings.	**Dynamic Schemas & Vector Retrieval:** Handles varying document payloads without DDL migrations. Built-in full-text indexing and dense vector arrays enable semantic query retrieval.
Application API Tier FastAPI (Python 3.13)	Asynchronous REST endpoints, JWT authentication, dual-tier RBAC, search federation.	**High Throughput & Strict Typing:** Python's `asyncio` allows parallel dispatch across PostgreSQL and MongoDB. Pydantic v2 validates requests before execution.
User Interface Tier Next.js 15 + React 19	Case management views, interactive relationship graph, semantic search dashboard.	**Modular Component Architecture:** Client-side routing and reactive UI state management styled with a Tailwind v4 graphite palette.

2. Architectural Trade-Off Analysis

Operating a polyglot persistence architecture introduces specific trade-offs that we addressed through explicit architectural decisions:

- **Eventual Consistency across Stores:** Relational tables store authoritative foreign keys, while MongoDB documents store cross-store reference strings (`case_id`). We do not perform distributed 2-phase commits across PostgreSQL and MongoDB; instead, the application-layer API validates entity existence before persisting documents, providing high write availability.
- **Unified Search Latency:** Rather than running sequential queries, FastAPI dispatches search requests to PostgreSQL and MongoDB in parallel using `asyncio.gather`, merging and ranking results before returning a

single JSON response.

- **Database Engine Portability:** To prevent evaluator lock-in, our database layer uses SQLAlchemy 2.0 with automated dialect detection. If a live PostgreSQL server is unreachable, the system automatically falls back to a local SQLite database with identical tables and constraints.